Ultimate Interview Prep Guide

Data Structures & Algorithms · Full-Stack MERN Development

150+ Questions · Detailed Answers · Code Examples · Pro Tips

Curated to help you crack top-tier software engineering interviews at product companies and startups alike.

2025 Edition · Covers: Arrays · Trees · Graphs · DP · React · Node.js · MongoDB · System Design

■ Table of Contents

PART 1 — Data Structures & Algorithms

1. Arrays & Strings

Q1. What is an array and what are its time complexities for common operations?

An array is a contiguous block of memory that stores elements of the same type. Access by index is O(1). Insertion/deletion at the end is O(1) amortised (dynamic arrays). Insertion/deletion at an arbitrary position is O(n) because elements must be shifted. Searching an unsorted array is O(n); a sorted array with binary search is O(log n).

■ *Tip: Always clarify whether the array is sorted — it changes your approach entirely.*

Q2. Find the two numbers in an array that add up to a target sum.

Use a Hash Set for O(n) time complexity. Iterate through the array; for each element x, check whether (target - x) already exists in the set. If yes, you found the pair. Otherwise add x to the set.

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        diff = target - n
        if diff in seen: return [seen[diff], i]
        seen[n] = i
```

■ *Tip: If the array is sorted, use the two-pointer technique for O(1) extra space.*

Q3. What is the sliding window technique? Give an example.

The sliding window technique maintains a contiguous subarray (window) between two pointers and expands/shrinks it instead of recomputing from scratch. It reduces $O(n^2)$ brute-force to O(n). Example: find the maximum sum subarray of size k — slide the window by adding the new element and removing the old one.

```
def max_sum_subarray(arr, k):
    win = sum(arr[:k]); best = win
    for i in range(k, len(arr)):
        win += arr[i] - arr[i-k]
        best = max(best, win)
    return best
```

Q4. How do you find the maximum product subarray?

Keep track of both the current maximum and current minimum products (because a negative × negative = positive). At each step: curr_max = max(num, curr_max*num, curr_min*num), curr_min = min(...). Update the global max at each step. Time: O(n), Space: O(1).

■ *Tip: The min product matters because multiplying two negatives gives a large positive.*

Q5. Explain the two-pointer technique with an example.

Two pointers start at opposite ends (or both at the start) and move toward each other based on a condition. Classic example — check if an array has a pair summing to target (sorted array): move left pointer right if sum is too small, right pointer left if too large. O(n) time, O(1) space.

■ *Tip: Two pointers are often the key to reducing $O(n^2)$ to $O(n)$ on sorted data.*

2. Linked Lists

Q6. What is the difference between a singly and doubly linked list?

A singly linked list has nodes with one 'next' pointer; traversal is one-directional and deletion requires tracking the previous node. A doubly linked list has both 'next' and 'prev' pointers, allowing O(1) deletion given a node reference and bidirectional traversal, at the cost of extra memory per node.

Q7. How do you detect a cycle in a linked list?

Use Floyd's Cycle Detection algorithm (tortoise and hare). Two pointers start at head; slow moves one step, fast moves two steps. If they ever meet, a cycle exists. If fast reaches null, no cycle. Time: O(n), Space: O(1).

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast: return True
    return False
```

Q8. How do you reverse a linked list in-place?

Use three pointers: prev (None), curr (head), and next_node. In each iteration, point curr.next to prev, then advance all three pointers. Return prev as the new head. Time: O(n), Space: O(1).

```
def reverse(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev, curr = curr, nxt
    return prev
```

Q9. How do you find the middle of a linked list?

Use the slow/fast pointer technique. Fast moves twice as fast as slow. When fast reaches the end, slow is at the middle. For even-length lists, this gives the second middle node, which is usually what is wanted.

■ *Tip: For merging, finding the middle and then reversing the second half is a common pattern.*

3. Stacks & Queues

Q10. What is a Stack and what are its key operations?

A stack follows LIFO (Last In, First Out). Core operations: push (add to top) O(1), pop (remove from top) O(1), peek/top (view top) O(1), isEmpty O(1). Common use cases: function call stack, undo operations, expression evaluation, DFS traversal.

■ *Tip: In Python, use a list as a stack: append() to push, pop() to pop.*

Q11. How do you design a stack that supports getMin() in O(1)?

Maintain an auxiliary min_stack alongside the main stack. Push a value onto min_stack only if it is ≤ the current minimum. On pop, if the popped value equals the top of min_stack, pop from min_stack too. getMin() returns min_stack's top. Time and space: O(1) per operation.

```
class MinStack:
    def __init__(self):
        self.stack=[]; self.min=[]
    def push(self,val):
        self.stack.append(val)
        self.min.append(val if not self.min else min(val,self.min[-1]))
    def pop(self):
        self.stack.pop(); self.min.pop()
    def getMin(self):
        return self.min[-1]
```

Q12. What is the difference between a Queue and a Deque?

A queue follows FIFO (First In, First Out) with enqueue at rear and dequeue at front. A deque (double-ended queue) allows insertion and deletion from both ends in O(1). Deque is used in sliding window maximum problems and implementing both stacks and queues.

■ *Tip: Python's collections.deque is optimal — O(1) for appendleft/popleft unlike list.*

---

4. Trees & Binary Search Trees

Q13. What are the four traversal methods of a binary tree?

Inorder (Left-Root-Right): gives sorted output for a BST. Preorder (Root-Left-Right): used to clone/serialize a tree. Postorder (Left-Right-Root): used for deletion and evaluating expression trees. Level-order (BFS): visits nodes level by level using a queue.

■ *Tip: Inorder of BST = sorted array. Memorise this — it's asked constantly.*

Q14. What is the difference between a Binary Tree and a Binary Search Tree?

A Binary Tree is any tree where each node has at most two children. A BST additionally satisfies the BST property: all nodes in the left subtree have values less than the root, and all nodes in the right subtree have values greater. This allows O(log n) average search, insert, and delete.

Q15. How do you find the height/depth of a binary tree?

Use recursion: the height of a node = 1 + max(height(left), height(right)). Base case: null node returns -1 (or 0). This runs in O(n) time since every node is visited once.

```
def height(root):
    if not root: return -1
    return 1 + max(height(root.left), height(root.right))
```

Q16. What is a balanced binary tree? How do you check if a tree is balanced?

A tree is balanced if for every node, the height difference between left and right subtrees is at most 1. Check recursively: compute height bottom-up; if any subtree's height difference > 1, return -1 as a sentinel. Overall time complexity: O(n).

Q17. What is the Lowest Common Ancestor (LCA) of two nodes?

The LCA of nodes p and q is the deepest node that is an ancestor of both. For a BST: if both p and q are smaller than root, LCA is in the left subtree; if both are larger, in the right; otherwise root is the LCA. For a general tree use recursion: if root equals p or q, root is LCA; otherwise recurse both sides and combine.

■ *Tip: LCA appears in distance between nodes, path queries, and range minimum queries.*

5. Graphs & BFS / DFS

Q18. What is the difference between BFS and DFS?

BFS (Breadth-First Search) explores nodes level by level using a queue. It finds the shortest path in unweighted graphs. Time and space: O(V+E). DFS (Depth-First Search) explores as deep as possible before backtracking, using a stack (or recursion). Useful for cycle detection, topological sort, connected components.

■ *Tip: BFS = shortest path. DFS = exhaustive exploration, cycle detection, topological sort.*

Q19. How do you detect a cycle in a directed graph?

Use DFS with a 'rec_stack' (recursion stack). Mark visited nodes and nodes currently on the call stack. If a DFS finds a node already in rec_stack, a cycle exists. Time: O(V+E).

```
def has_cycle(graph, V):
    visited=[False]*V; rec=[False]*V
    def dfs(v):
        visited[v]=rec[v]=True
        for u in graph[v]:
            if not visited[u] and dfs(u): return True
            elif rec[u]: return True
        rec[v]=False; return False
    return any(not visited[v] and dfs(v) for v in range(V))
```

Q20. What is Topological Sorting? When is it used?

Topological sort orders nodes in a DAG (Directed Acyclic Graph) such that for every directed edge u→v, u comes before v. It is used in task scheduling, build systems (Makefile), course prerequisites,

and package dependency resolution. Implemented via DFS post-order or Kahn's algorithm (BFS with in-degree).

Q21. Explain Dijkstra's algorithm.

Dijkstra finds the shortest path from a source node to all others in a weighted graph with non-negative edges. It uses a min-heap (priority queue). Extract the minimum distance node, relax its neighbours, and push updated distances into the heap. Time: O((V+E) log V) with a binary heap.

■ *Tip: Dijkstra does NOT work with negative edge weights — use Bellman-Ford instead.*

6. Sorting & Searching Algorithms

Q22. Compare Merge Sort and Quick Sort.

Merge Sort: divides array in half, sorts each half, then merges. Always O(n log n), stable, but requires O(n) extra space. Quick Sort: picks a pivot, partitions elements around it, recurses. Average O(n log n), worst O(n^2) (bad pivot). In-place (O(log n) stack space), not stable. In practice Quick Sort is faster due to cache efficiency.

■ *Tip: Merge Sort is preferred for linked lists; Quick Sort for arrays.*

Q23. How does Binary Search work? What are its prerequisites?

Binary Search requires a sorted array. It compares the middle element with the target. If equal, return mid. If target is smaller, search the left half; if larger, search the right half. Each step halves the search space. Time: O(log n), Space: O(1) iterative.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr)-1
    while lo <= hi:
        mid = (lo+hi)//2
        if arr[mid]==target: return mid
        elif arr[mid]<target: lo=mid+1
        else: hi=mid-1
    return -1
```

Q24. What is Counting Sort? When should you use it?

Counting Sort is a non-comparison sort that counts occurrences of each element and reconstructs the sorted array. Time: O(n+k) where k is the range of values. It is optimal when k is small relative to n (e.g., sorting student grades 0-100 or single characters). Not suitable for large or floating-point ranges.

■ *Tip: Counting Sort is the basis for Radix Sort, which can sort integers in O(nk) time.*

7. Dynamic Programming

Q25. What is Dynamic Programming? What are its two approaches?

DP solves problems by breaking them into overlapping subproblems and storing results to avoid recomputation (memoization). Two approaches: (1) Top-Down (Memoization) — recursive + cache, intuitive but stack overhead. (2) Bottom-Up (Tabulation) — iterative, fills a table from base cases up, usually more space-efficient.

■ *Tip: If you can define a recurrence relation, you can implement DP.*

Q26. Explain the Longest Common Subsequence (LCS) problem.

LCS finds the longest sequence present in both strings in the same order (not necessarily contiguous). Define dp[i][j] = LCS of first i chars of s1 and first j chars of s2. If s1[i-1]==s2[j-1]: dp[i][j]=dp[i-1][j-1]+1, else dp[i][j]=max(dp[i-1][j], dp[i][j-1]). Answer is dp[m][n]. Time O(mn), Space O(mn) or O(min(m,n)) optimised.

■ *Tip: LCS is the foundation for diff tools (git diff) and DNA sequence alignment.*

Q27. Explain the 0/1 Knapsack problem.

Given weights and values of n items and a capacity W, maximise the total value without exceeding W. Each item can only be included once (0/1). DP: dp[i][w] = max value using first i items with capacity w. Transition: dp[i][w] = max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i]) if wt[i] <= w.

```
def knapsack(W, wt, val, n):
    dp=[[0]*(W+1) for _ in range(n+1)]
    for i in range(1,n+1):
        for w in range(W+1):
            dp[i][w]=dp[i-1][w]
            if wt[i-1]<=w:
                dp[i][w]=max(dp[i][w],dp[i-1][w-wt[i-1]]+val[i-1])
    return dp[n][W]
```

Q28. What is the Fibonacci sequence DP solution?

Naive recursion is O(2^n). DP memoization stores results: memo[n] = fib(n-1) + fib(n-2). Bottom-up tabulation uses two variables: a, b = 0, 1; iterate n times; a, b = b, a+b. Result: O(n) time, O(1) space.

8. Hashing & Hash Maps

Q29. How does a Hash Map work internally?

A hash map uses a hash function to map keys to bucket indices. Each bucket stores key-value pairs. Collisions (two keys mapping to same index) are handled by chaining (each bucket is a linked list) or open addressing (probe for the next empty slot). Average O(1) for get/put/delete; O(n) worst case.

Q30. What is the load factor and when does rehashing occur?

Load factor = (number of entries) / (number of buckets). When it exceeds a threshold (typically 0.75 in Java's HashMap), rehashing occurs: a new larger array is allocated and all entries are re-inserted. This is O(n) but amortised O(1) per insertion. Python's dict rehashes at ~2/3 capacity.

■ *Tip: A lower load factor means fewer collisions but more memory usage.*

9. Recursion & Backtracking

Q31. What is backtracking? How does it differ from brute force?

Backtracking is a refined brute force that abandons (prunes) a partial solution as soon as it is determined to be invalid, rather than generating all possibilities. It builds candidates incrementally and undoes choices (backtracks) when a dead end is reached. Examples: N-Queens, Sudoku solver, permutations, subsets.

■ *Tip: Think of backtracking as DFS on the decision tree with pruning.*

Q32. Generate all permutations of a given string.

Use backtracking: choose a character at each position from remaining characters, recurse for the rest, then unchoose (backtrack). Time: O(n × n!) because there are n! permutations each of length n.

```
def permutations(s):
    res=[]
    def bt(path, remaining):
        if not remaining: res.append(path); return
        for i,c in enumerate(remaining):
            bt(path+c, remaining[:i]+remaining[i+1:])
    bt('', s); return res
```

10. Complexity Analysis (Big-O)

Q33. What is Big-O notation? List common complexities from best to worst.

Big-O describes an algorithm's worst-case time or space growth relative to input size n. Common complexities: O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, O(n^2) quadratic, O(2^n) exponential, O(n!) factorial.

■ *Tip: O(n log n) is the best achievable for comparison-based sorting.*

Q34. What is amortised analysis? Give an example.

Amortised analysis averages the cost over a sequence of operations, even if individual operations are expensive. Example: dynamic array's append is usually O(1), but occasionally O(n) when the array doubles. Averaged over n appends, cost is O(1) amortised because doublings become less frequent.

■ *Tip: Stack with doubling strategy, Python lists, and hash table rehashing all use amortised analysis.*

PART 2 — Full-Stack MERN Development

11. JavaScript & ES6+

Q35. What is the difference between var, let, and const?

var is function-scoped and hoisted (declared variables initialised to undefined). let is block-scoped and not initialised on hoist (Temporal Dead Zone). const is block-scoped, not reinitializable (but object properties can still be mutated). Best practice: prefer const by default, let when you need reassignment, avoid var.

■ *Tip: Use const and let; var is legacy and can cause bugs from unexpected hoisting.*

Q36. Explain closures in JavaScript.

A closure is a function that retains access to variables from its outer (enclosing) scope even after the outer function has returned. This enables data privacy and function factories. Example: a counter factory returns an inner function that increments a private count variable.

```
function makeCounter() {
let count = 0;
return () => ++count;
}
const c = makeCounter();
console.log(c()); // 1
console.log(c()); // 2
```

■ *Tip: Closures power module patterns, event listeners, and React hooks.*

Q37. What is the Event Loop in JavaScript?

JavaScript is single-threaded. The event loop manages async operations: the call stack runs synchronous code; when it is empty, the event loop picks tasks from the task queue (macrotasks: setTimeout, setInterval) or microtask queue (Promises, queueMicrotask). Microtasks are always processed before the next macrotask.

■ *Tip: Promise.resolve().then() runs before setTimeout(() => {}, 0) — microtasks have priority.*

Q38. What are Promises and async/await?

A Promise represents the eventual result of an async operation with states: pending, fulfilled, rejected. .then()/.catch() chain handlers. async/await is syntactic sugar that makes async code look synchronous. An async function always returns a Promise. await pauses execution until the Promise resolves.

```
async function fetchUser(id) {
try {
    const res = await fetch(`/api/users/${id}`);
    const data = await res.json();
    return data;
} catch (err) {
```

```
    console.error(err);
}
}
```

Q39. What is the difference between == and ===?

== performs type coercion before comparison (e.g., '5' == 5 is true). === (strict equality) compares both value and type without coercion (e.g., '5' === 5 is false). Always use === in production code to avoid unexpected type conversion bugs.

■ *Tip: null == undefined is true, but null === undefined is false.*

Q40. Explain prototypal inheritance in JavaScript.

Every JavaScript object has an internal [[Prototype]] link to another object (its prototype). When a property is not found on an object, the engine walks up the prototype chain until it finds it or reaches null. ES6 classes are syntactic sugar over this prototype-based system. Object.create() lets you set prototypes explicitly.

■ *Tip: Use Object.getPrototypeOf(obj) to inspect the prototype chain.*

12. React.js

Q41. What is the Virtual DOM and how does React use it?

The Virtual DOM is an in-memory JavaScript representation of the real DOM. When state changes, React re-renders the component into a new Virtual DOM, diffs it against the previous one (reconciliation), and applies only the minimal set of changes to the real DOM. This batching and diffing makes updates faster.

■ *Tip: The reconciliation algorithm (Fiber) assigns priority to updates — high-priority ones interrupt lower ones.*

Q42. What are React Hooks? Name the most important ones.

Hooks (introduced in v16.8) let functional components use state and lifecycle features. Most important: useState (local state), useEffect (side effects/lifecycle), useContext (consume context), useRef (mutable ref without re-render), useMemo (memoize expensive values), useCallback (memoize functions), useReducer (complex state logic), custom hooks (reusable stateful logic).

■ *Tip: Custom hooks should start with 'use' — this is a convention enforced by lint rules.*

Q43. What is the difference between useEffect with no deps, [], and [dep]?

useEffect(() => {}) runs after every render (no deps array). useEffect(() => {}, []) runs only once after the initial mount (empty deps = componentDidMount). useEffect(() => {}, [dep]) runs on mount and whenever dep changes (componentDidUpdate for that dep). The cleanup function (returned from effect) runs before the next effect and on unmount.

```
useEffect(() => {
const sub = subscribe(id);
```

```
return () => sub.unsubscribe(); // cleanup
}, [id]);
```

Q44. What is React Context and when should you use it?

Context provides a way to pass data through the component tree without prop-drilling at every level. Create context with React.createContext(), wrap the tree in a Provider, and consume with useContext(). Best for: theming, locale/i18n, auth state, user preferences. For complex state logic with many actions, prefer Redux or Zustand as Context can cause unnecessary re-renders.

■ *Tip: Splitting contexts (ThemeContext, AuthContext) limits re-render scope.*

Q45. What is the difference between controlled and uncontrolled components?

Controlled components: form element value is controlled by React state via value prop + onChange handler. React is the 'single source of truth'. Uncontrolled components: form data is managed by the DOM itself; you use a ref to read the value when needed. Controlled components are preferred for validation and complex form interactions; uncontrolled for simple cases or integrating non-React libraries.

Q46. What is React.memo and when should you use it?

React.memo is a HOC that shallowly compares previous and new props of a functional component. If props haven't changed, it skips re-rendering. Use it for pure components that receive the same props often. Pair with useCallback (for function props) and useMemo (for object props) to prevent reference changes.

■ *Tip: Profile first (React DevTools Profiler) before optimising — premature memoisation adds complexity.*

Q47. What is code splitting in React and how do you implement it?

Code splitting breaks the bundle into smaller chunks loaded on demand, reducing initial load time. Implement with React.lazy() and Suspense: lazy-load a component so it is only fetched when rendered. Combined with React Router, each route can be a separate chunk.

```
const Dashboard = React.lazy(() => import('./Dashboard'));
<Suspense fallback={<Spinner />}>
<Dashboard />
</Suspense>
```

Q48. What is Redux and what problem does it solve?

Redux is a predictable state container. It solves the problem of managing shared state across many components without prop-drilling. Core concepts: Store (single source of truth), Action (plain object describing change), Reducer (pure function: (state, action) => newState), Dispatch (sends actions to the store), Selector (reads derived state). Redux Toolkit (RTK) simplifies boilerplate with createSlice and createAsyncThunk.

■ *Tip: Context API is fine for infrequently updated global state; Redux shines for high-frequency updates and complex logic.*

13. Node.js & Express.js

Q49. What is Node.js and how does it handle concurrent requests without threads?

Node.js is a JavaScript runtime built on Chrome's V8 engine with a non-blocking, event-driven I/O model. It uses a single-threaded event loop backed by libuv (a C library with a thread pool for I/O). When an async I/O call (file read, DB query) is made, Node delegates it to libuv, continues processing other requests, and invokes the callback once the I/O completes. This makes it ideal for I/O-bound workloads.

■ *Tip: Node is not ideal for CPU-intensive tasks — they block the event loop. Use worker_threads or child_process.*

Q50. What is middleware in Express.js?

Middleware are functions with signature (req, res, next) that execute during the request-response cycle. They can modify req/res objects, end the cycle, or call next() to pass control to the next middleware. Types: application-level (app.use), router-level, error-handling (4 args: err, req, res, next), built-in (express.json), third-party (morgan, cors, helmet).

```
// Logger middleware
app.use((req, res, next) => {
console.log(`${req.method} ${req.url}`);
next();
});
```

Q51. How do you handle errors in Express?

Define an error-handling middleware with four parameters (err, req, res, next) placed after all other routes. In async route handlers, wrap code in try-catch and pass errors to next(err). Use centralized error classes and map them to HTTP status codes. Libraries like express-async-errors can automatically catch async errors.

```
app.use((err, req, res, next) => {
const status = err.statusCode || 500;
res.status(status).json({ error: err.message });
});
```

Q52. What is the difference between process.nextTick and setImmediate?

process.nextTick queues a callback to run before the next event loop iteration (before any I/O). setImmediate runs in the check phase of the current event loop iteration, after I/O callbacks. nextTick has higher priority and can starve the event loop if overused.

■ *Tip: Use setImmediate to break up CPU work across iterations; use nextTick only when you need to run before I/O.*

14. MongoDB & Mongoose

Q53. What is MongoDB and how does it differ from SQL databases?

MongoDB is a NoSQL document database that stores data as BSON (Binary JSON) documents in collections. Differences from SQL: schema-less (flexible structure), horizontal scaling with sharding, no joins (embed related data or use $lookup), suitable for hierarchical/denormalized data. SQL databases enforce schema, ACID transactions, and are better for complex relational queries.

■ *Tip: MongoDB 4.0+ supports multi-document ACID transactions, narrowing the gap with SQL.*

Q54. What are indexes in MongoDB and why are they important?

Indexes are data structures (B-tree by default) that speed up query execution by avoiding full collection scans. Without an index, MongoDB must examine every document (O(n)). With an index, lookup is O(log n). Types: single field, compound, text (full-text search), geospatial, hashed. Use explain() to see whether a query uses an index (IXSCAN vs COLLSCAN).

```
// Create a compound index
db.users.createIndex({ email: 1, createdAt: -1 });
```

■ *Tip: Over-indexing hurts write performance — index only fields used in frequent queries.*

Q55. What is Mongoose and what does it add to MongoDB?

Mongoose is an ODM (Object Data Modeling) library for Node.js that provides: schema definition with types/validation, model abstraction, middleware (pre/post hooks), virtuals, population (joining documents), and connection management. It enforces structure on MongoDB's schema-less documents at the application layer.

```
const userSchema = new mongoose.Schema({
name: { type: String, required: true },
email: { type: String, unique: true }
});
const User = mongoose.model('User', userSchema);
```

Q56. Explain MongoDB aggregation pipeline.

The aggregation pipeline processes documents through a series of stages, each transforming the data. Common stages: $match (filter), $group (group + aggregate like SQL GROUP BY), $project (reshape docs), $sort, $limit, $skip, $lookup (left outer join), $unwind (flatten arrays). Pipelines are highly efficient when using indexes in early $match stages.

■ *Tip: Always put $match and $sort early in the pipeline to leverage indexes and reduce the working set.*

15. REST APIs & Authentication

Q57. What are the principles of RESTful API design?

REST (Representational State Transfer) principles: (1) Uniform Interface — consistent resource-based URLs (nouns not verbs: /users not /getUsers). (2) Stateless — no client state stored on server; each request is self-contained. (3) Client-Server separation. (4) Cacheable — responses indicate cacheability. (5) Layered System. HTTP verbs map to CRUD: GET=read, POST=create, PUT=update, PATCH=partial update, DELETE=remove.

■ *Tip: Use HTTP status codes correctly: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error.*

Q58. What is JWT and how does token-based authentication work?

JWT (JSON Web Token) is a compact, self-contained token with three parts: Header.Payload.Signature (Base64 encoded). Flow: user logs in → server verifies credentials → server signs a JWT with a secret and returns it → client stores JWT (localStorage or httpOnly cookie) → client sends JWT in Authorization: Bearer header → server verifies signature and extracts user info without DB lookup.

■ *Tip: Store JWTs in httpOnly cookies to prevent XSS. Use short expiry (15min) + refresh tokens for security.*

Q59. What is the difference between authentication and authorization?

Authentication verifies who you are (identity) — e.g., logging in with email/password or OAuth. Authorization determines what you are allowed to do (permissions) — e.g., only admins can delete users. In code: authentication middleware validates the JWT/session; authorization middleware checks user roles/permissions.

```
// Auth middleware
const protect = (req,res,next) => {
const token = req.headers.authorization?.split(' ')[1];
if(!token) return res.status(401).json({error:'Unauthorized'});
req.user = jwt.verify(token, process.env.JWT_SECRET);
next();
};
```

Q60. What is CORS and how do you handle it in Express?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks requests from a different origin (protocol + domain + port) by default. Configure the server to send CORS headers allowing specific origins. In Express, use the cors middleware. Pre-flight OPTIONS requests are handled automatically.

```
const cors = require('cors');
app.use(cors({
origin: 'https://myapp.com',
credentials: true
}));
```

16. System Design & Architecture

Q61. What is the MVC pattern? How does it map to MERN?

MVC (Model-View-Controller) separates concerns: Model manages data/logic, View handles UI, Controller mediates. In MERN: MongoDB = Model (data layer), React = View (UI), Express + Node.js = Controller (business logic + routing). This separation makes code maintainable, testable, and scalable.

■ *Tip: In modern React apps, the Controller role is often handled by API calls + state managers (Redux, Zustand).*

Q62. What is horizontal vs vertical scaling?

Vertical scaling (scale up): add more resources (CPU, RAM) to a single server. Simple but has a hardware limit and single point of failure. Horizontal scaling (scale out): add more server instances behind a load balancer. More complex (requires stateless servers, distributed sessions), but nearly unlimited scalability and fault tolerance. MERN apps scale horizontally by using JWTs (stateless auth) and MongoDB Atlas sharding.

■ *Tip: Stateless APIs (JWT, not server sessions) are a prerequisite for horizontal scaling.*

Q63. What is caching and how would you implement it in a MERN app?

Caching stores frequently accessed data in fast memory to reduce DB load and latency. Levels: (1) Browser cache (Cache-Control headers), (2) CDN for static assets, (3) Server-side with Redis. In Express + MongoDB: cache DB query results in Redis with a TTL. Invalidate cache on write operations. Common patterns: Cache-Aside, Write-Through, Write-Behind.

```
const cached = await redis.get(key);
if(cached) return JSON.parse(cached);
const data = await User.find(query);
await redis.setex(key, 3600, JSON.stringify(data));
return data;
```

Q64. What are microservices and how do they differ from monoliths?

A monolith is a single deployable unit containing all features. Easy to develop initially but hard to scale and maintain as it grows. Microservices decompose the app into small, independent services (user service, order service) that communicate via REST or message queues (RabbitMQ, Kafka). Each service has its own database and can be deployed and scaled independently.

■ *Tip: Start with a monolith; extract microservices only when you have clear scaling or team separation needs.*

17. Advanced Topics & Best Practices

Q65. What is WebSocket and when would you use it over REST?

WebSocket provides a persistent, full-duplex communication channel over a single TCP connection. Unlike REST (request-response), WebSocket allows the server to push data to the client at any time. Use WebSocket for real-time features: chat applications, live notifications, collaborative editing, live dashboards, multiplayer games. In Node.js, use the ws library or Socket.io.

■ *Tip: Socket.io adds fallback to long-polling for environments where WebSockets are blocked.*

Q66. How do you optimise the performance of a React application?

Key strategies: (1) Code splitting with React.lazy + Suspense to reduce bundle size. (2) Memoisation: React.memo, useMemo, useCallback to prevent unnecessary re-renders. (3) Virtualise

long lists with react-window. (4) Lazy-load images (loading='lazy'). (5) Debounce/throttle expensive operations like search input. (6) Use production builds (minified). (7) Analyse bundle with webpack-bundle-analyzer. (8) Use Lighthouse for auditing.

■ *Tip: Profile in React DevTools Profiler before optimising — identify actual bottlenecks.*

Q67. What is server-side rendering (SSR) and how does it differ from CSR?

CSR (Client-Side Rendering): browser downloads minimal HTML, then JS fetches data and renders the page. Poor initial load and SEO. SSR: server renders the full HTML page before sending to the browser. Faster Time to First Contentful Paint and better SEO. Next.js is the standard SSR framework for React, offering SSR, SSG (Static Site Generation), and ISR (Incremental Static Regeneration).

■ *Tip: Use SSG for mostly static content (blogs) and SSR for personalised/dynamic pages.*

Q68. What are environment variables and why are they important?

Environment variables store configuration and secrets (API keys, DB URIs, JWT secrets) outside source code. They allow the same codebase to run in development/staging/production with different configs. In Node.js, use the dotenv package to load .env files. Never commit .env to version control. In production, inject env vars through the platform (Heroku Config Vars, AWS Parameter Store).

```
require('dotenv').config();
const DB_URI = process.env.MONGO_URI;
```

Q69. What is CI/CD and how would you set it up for a MERN app?

CI (Continuous Integration) automatically builds and tests code on every push. CD (Continuous Delivery/Deployment) automatically deploys passing builds. For MERN: use GitHub Actions to run ESLint, Jest/Mocha tests, build the React app, then deploy backend to AWS/Heroku/Render and frontend to Vercel/Netlify. Docker containers ensure consistency across environments.

■ *Tip: A good CI pipeline catches bugs before they reach production, enabling confident frequent releases.*

Q70. What security best practices should you follow in a MERN application?

Key practices: (1) Hash passwords with bcrypt (never store plaintext). (2) Use JWT with short expiry + refresh tokens. (3) Validate and sanitize all inputs (express-validator, mongoose validation). (4) Prevent NoSQL injection by sanitizing MongoDB operators (express-mongo-sanitize). (5) Use Helmet.js to set secure HTTP headers. (6) Rate limiting (express-rate-limit) to prevent brute-force attacks. (7) Use HTTPS everywhere. (8) Store secrets in env vars, never in code.

■ *Tip: OWASP Top 10 is the security checklist every developer should know.*

Pro Interview Tip: Always think out loud. Interviewers want to see your reasoning process, not just the final answer. For DSA problems: clarify constraints → write examples → discuss approach → code → test with edge cases. For system design: scope the problem → high-level design → deep dive → trade-offs.

Good luck with your interviews! ■